

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 7: RISC-V Part 2

Instructors: Justin Yokota

Agenda

- Hailstone
- RISC-V Arithmetic Operators
- RISC-V Control Flow
- RISC-V Memory Operations

Agenda

- **Hailstone**
- RISC-V Arithmetic Operators
- RISC-V Control Flow
- RISC-V Memory Operations

Hailstone Sequence

The Hailstone sequence starts with a number N , then repeatedly does the following:

- If N is 1, terminate
- If N is odd, set N to $3N+1$
- If N is even, set N to $N/2$

Let's write a RISC-V program to compute a hailstone sequence!

How are you doing on the question?

CS 61C

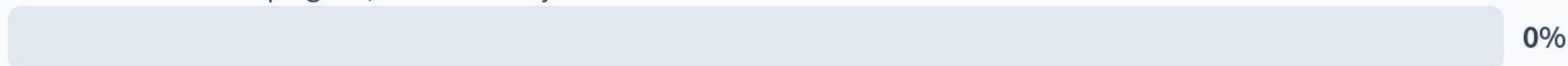
Summer 2026



I don't have a working solution yet, but have made good progress



I haven't made much progress, but have a way forward



I don't know what's going on, and need more guidance before I can start



Agenda

- Hailstone
- **RISC-V Arithmetic Operators**
- RISC-V Control Flow
- RISC-V Memory Operations

Arithmetic Operations in RISC-V

- RISC-V has many arithmetic operations:
 - `add rd rs1 rs2`: Add
 - `sub rd rs1 rs2`: Subtract
 - `and rd rs1 rs2`: Bitwise AND
 - `or rd rs1 rs2`: Bitwise OR
 - `xor rd rs1 rs2`: Bitwise XOR
 - `sll rd rs1 rs2`: Shift Left Logical
 - `srl rd rs1 rs2`: Shift Right Logical
 - `sra rd rs1 rs2`: Shift Right Arithmetic
- Each one has a corresponding immediate version (ex. `xori rd rs1 imm` runs a bitwise XOR on a register and immediate)

Sign- and Zero-extending

There are two main representation schemes used: unsigned numbers, and 2's complement

For unsigned numbers: 8-bit $0xEF = 239$. $239 \rightarrow 32$ bits is $0x000000EF$. General rule: Fill the top bits with 0s (Zero-extension).

For signed numbers: 8-bit $0xEF = -17$. $-17 \rightarrow 32$ bits is $0xFFFF FFEF$.

General rule: Fill the top bits with the *most significant bit* of the number (Sign-extension).

Arithmetic Operations in RISC-V

- RISC-V has many arithmetic operations:
 - `add rd rs1 rs2`: Add
 - `sub rd rs1 rs2`: Subtract
 - `and rd rs1 rs2`: Bitwise AND
 - `or rd rs1 rs2`: Bitwise OR
 - `xor rd rs1 rs2`: Bitwise XOR
 - `sll rd rs1 rs2`: Shift Left Logical
 - `srl rd rs1 rs2`: Shift Right Logical (Zero extend)
 - `sra rd rs1 rs2`: Shift Right Arithmetic (Sign extend)
- Each one has a corresponding immediate version (ex. `xori rd rs1 imm` runs a bitwise XOR on a register and immediate)

Arithmetic Operations that AREN'T in RISC-V

- Some instructions aren't present in RISC-V:
 - Bitwise NOT
 - Exists as a **pseudoinstruction**; can be run by doing a bitwise XOR with $0xFFFF\ FFFF = -1$
 - `li rd imm`: Pseudoinstruction that sets `rd` to the given immediate value.
 - Can be run by using `addi` (and `lui`; see Lecture 9)
 - Multiplication, Division, Mod
 - Circuit is significantly more complicated than bitwise ops or addition.
 - Exists as an *extension* of RISC-V, but is not by itself in base RISC-V
 - For multiplying/dividing by powers of 2, we can use shifts instead
 - Float addition/subtraction/etc.
 - Much more complicated circuit, so also exists as an extension.

Agenda

- Hailstone
- RISC-V Arithmetic Operators
- **RISC-V Control Flow**
- RISC-V Memory Operations

Control Flow

- For most instructions, the program continues execution by moving to the next instruction in the file.
- Any instructions that don't move to the next instruction affect **control flow**.
- In C:
 - if
 - while
 - for
 - function calls

Labels

- In RISC-V, every control flow operation tries to move to a specified instruction in memory.
 - C while loop: "Step 5: Repeat steps 3 and 4 until XXX"
 - RISC-V loop: "Step 5: If XXX is false, go to step 3"
- Labels identify a specific instruction in the code so we can reference it with a control flow operation

Control Flow Example

`beq rs1 rs2 Label:` If `rs1 == rs2`, jump to Label. (Branch if Equal)
`j Label:` Jump to Label (unconditionally)

```

    li x5 5
    li x6 0
Loop: add x6 x6 x5
      addi x5 x5 -1
      beq x5 x0 Done
      j Loop
Done: ...
```

RISC-V Control Flow Operations

List of branch instructions:

- `beq rs1 rs2 Label:` Branch if EQual
- `bne rs1 rs2 Label:` Branch if Not Equal
- `blt rs1 rs2 Label:` Branch if Less Than (signed) ($rs1 < rs2$)
- `bge rs1 rs2 Label:` Branch if Greater or Equal (signed)
- `bltu rs1 rs2 Label:` Branch if Less Than (unsigned)
- `bgeu rs1 rs2 Label:` Branch if Greater or Equal (unsigned)
- `bgt`, `bgtu`, `ble`, and `bleu` are pseudoinstructions (can make them by reversing inputs of existing instructions)

Quick Aside: Register Names

Each register has a name associated with them. They signal the most common use case for that register.

For now:

- x10 and x11 are a0 and a1 (Argument registers)
- x5 and x6 are t0 and t1 (Temporary registers)
- x8 is s0 (Saved register)

Writing Hailstone

Write in Venus a program that starts with a value in `a0`, and prints out the hailstone sequence starting from that value.

- You may use `t0` and `t1` as "free cells" for temporary values, and `s0` for more permanent variables

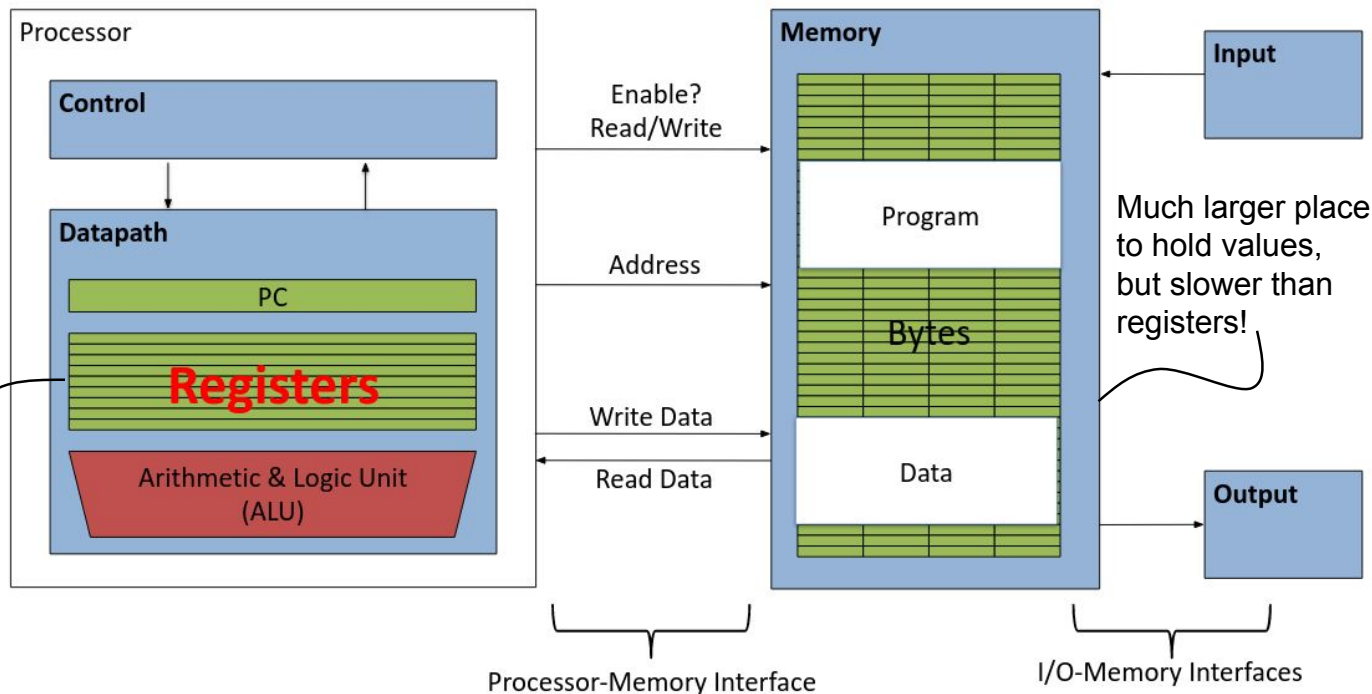
To print an integer in `a0` out, you can use the following snippet (modifies `a1` and `a0`, so be sure to save `a0` in `s0`!):

```
add a1 a0 x0
li a0 1
ecall
li a0 11
li a1 10
ecall
```

Agenda

- Hailstone
- RISC-V Arithmetic Operators
- RISC-V Control Flow
- **RISC-V Memory Operations**

Data Transfer: Load from and Store to memory



Review: How memory works

- On a 32-bit system, main memory contains 2^{32} bytes. Every 32-bit number acts as the **address** of one byte.
- 4 bytes together make a **word**. In order to store a word in memory, we cut the word up into 4 bytes, then store those bytes in consecutive addresses. In order to read a word, we read 4 consecutive addresses, then stitch those bytes back together
- RISC-V uses little-endian to store data: The least significant byte gets stored at the lowest address.
- By convention we say that a word is stored at its lowest address
 - Ex. The word stored at address 0x1000 is composed of the bytes 0x1000, 0x1001, 0x1002, and 0x1003

Agenda

- Main Memory
- **Data Transfer Instructions**
- C code examples

Load Word

Load Word syntax:

- `lw rd imm(rs1)`

Means: Compute $\text{imm} + \text{rs1}$, then load the 4 bytes at that address into `rd`

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Load Word

Example: `lw x10 12(x5)` if `x5` is `0x100`

- $0x100 + 12 = 0x10C$
- Bytes at `0x10C-0x10F` are `0x53`, `0x42`, `0x56`, `0x00`
- Since RISC-V is little-endian, this is the 32-bit value `0x0056 4253`
- So register `x10` will now store `0x0056 4253`

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Store Word

Store Word syntax:

- `sw rs2 imm(rs1)`

Means: Compute `imm+rs1`, then store the 4 bytes of `rs2` into that address.

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Store Word

Example: `sw x10 0(x5)` if `x5` is `0x100`, `x10` is `0x1234 5678`

- $0x100 + 0 = 0x100$
- Since RISC-V is little-endian, the 32-bit value `0x1234 5678` gets split into bytes `0x78 0x56 0x34 0x12`
- So the bytes in memory `0x100`, `0x101`, `0x102`, and `0x103` get set to `0x78`, `0x56`, `0x34`, and `0x12`, respectively.

Byte (0x)	78	56	34	12	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Loading and Storing Bytes

In addition to word data transfers (lw, sw), RISC-V has byte data transfers:

- `lb rd imm(rs1)`
- `sb rs2 imm(rs1)`

Load and store one byte instead of a full word

Problem: If registers contain 4 bytes, how do we load/store only 1 byte?

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Store Byte

Example: `sb x10, 0(x5)` if `x5` is `0x100`, `x10` is `0x1234 5678`

For `sb`, we store the *least significant byte*.

In the above example, `0x78` is the LSB.

So `x100` gets set to `0x78`

Byte (0x)	78	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Load Byte

Example: `lb x10, 0(x5)` if `x5` is `0x100`

For `lb`, we *extend* the numeric value to a full 32 bits.

In the above example, we load the number `0xEF`. What 32-bit number has the same numeric value as `0xEF`?

Answer: It depends on your representation scheme

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Loading and Storing Bytes

In addition to word data transfers (lw, sw), RISC-V has byte data transfers:

- `lb rd imm(rs1)` -> sign extend the byte
- `lbu rd imm(rs1)` -> zero extend the byte
- `sb rs2 imm(rs1)` -> store least significant byte only

Load and store one byte instead of a full word

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

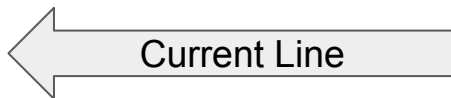
```
li x11, 0x93F5
```

```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```

Example: What is in x12?

```
li x11, 0x93F5
```



```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```

Register	Value
x11	0x00000000
x12	0x00000000
x5	0x00000100

Random
garbage since
data isn't set

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

```
li x11, 0x93F5
```



```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```

Register	Value
x11	0x000093F5
x12	0x00000000
x5	0x00000100

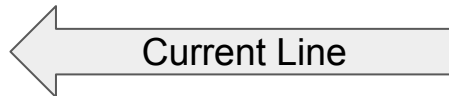
Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

```
li x11, 0x93F5
```

```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```



Register	Value
x11	0x000093F5
x12	0x00000000
x5	0x00000100

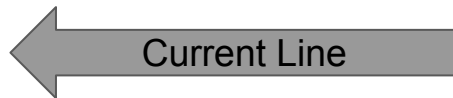
Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

```
li x11, 0x93F5
```

```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```



Register	Value
x11	0x000093F5
x12	0x00000000
x5	0x00000100

Little-Endian
order

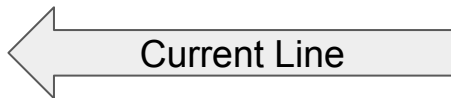
Byte (0x)	F5	93	00	00	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

```
li x11, 0x93F5
```

```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```



Register	Value
x11	0x000093F5
x12	0x00000000
x5	0x00000100

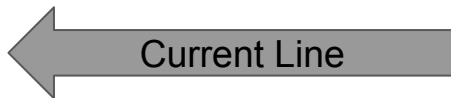
Byte (0x)	F5	93	00	00	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

```
li x11, 0x93F5
```

```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```



Register	Value
x11	0x000093F5
x12	0xFFFFFFFF93
x5	0x00000100

Sign-extend: Top
bit of 0x93 is 1,
so fill with 1s

Byte (0x)	F5	93	00	00	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

x2: The Stack Pointer (sp)

The register x2 is set to a special value at the start of a program. Normally, it's used to keep track of the **stack**.

- Everything above sp when you start the program is **reserved**
- Everything below sp is **unallocated**
- Everything between sp and original location of sp is useable space for you.

To use the stack:

- Move sp down
- Store your data in the stack

Example Stack use

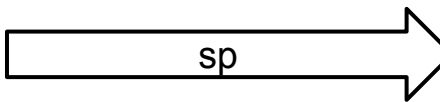
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Example Stack use

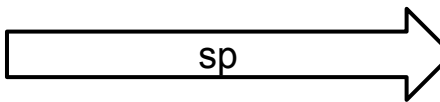
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Example Stack use

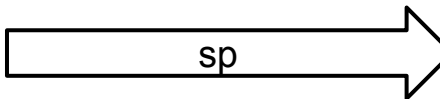
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Example Stack use

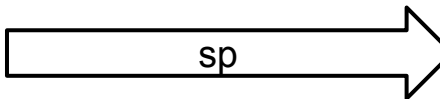
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Example Stack use

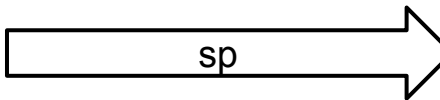
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0	EF	BE	AD	DE
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Example Stack use

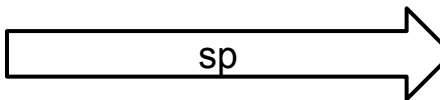
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0	EF	BE	AD	DE
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Writing Reverse Hailstone

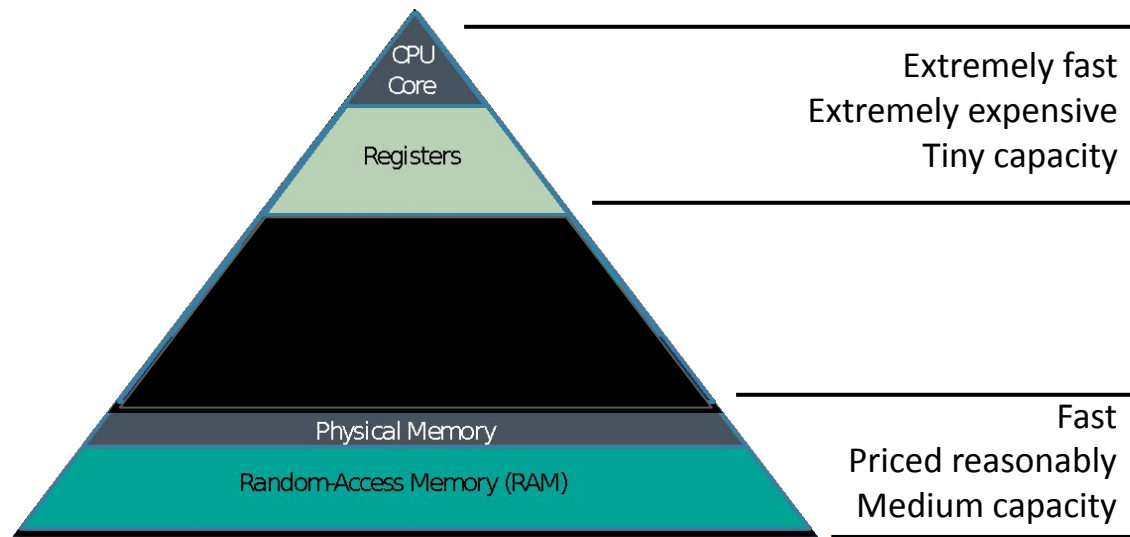
Write in Venus a program that starts with a value in `a0`, and prints out the hailstone sequence starting from that value **in reverse order**.

- You may use `t0` and `t1` as "free cells" for temporary values, and `s0` and `s1` for more permanent variables
- You may use the `sp` as defined previously (only space below where `sp` was when you start the program, and above where `sp` is now)

Why we need so many registers

- As the previous example showed, it's possible to write RISC-V with only a sp and three temporary registers
- Why do we have 32 registers?

RISC-V Guiding Philosophy

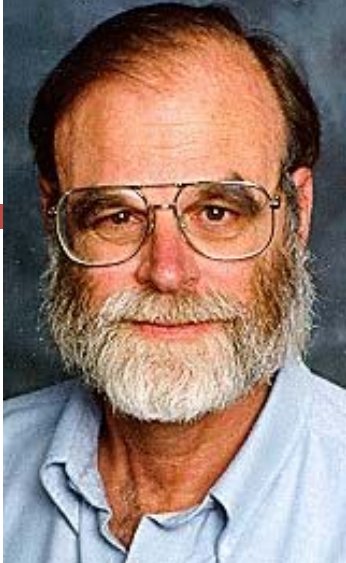


Speed of Registers vs Memory

- Given that
 - Registers: 32 words (128 Bytes)
 - Memory (DRAM): Billions of bytes (2 GB to 96 GB on laptop)
- and physics dictates...
 - Smaller is faster
- How much faster are registers than DRAM??
 - About 50-500 times faster!
(in terms of latency of one access - tens of ns)
 - But subsequent words come every few ns

Jim Gray's Storage Latency Analogy: How Far Away is the Data?

CS 61C



Jim Gray
Turing Award
B.S. Cal 1966
Ph.D. Cal
1969

100 Memory



Sacramento

1 Registers



1 min

And in Conclusion...

- Memory is byte-addressable, but **lw** and **sw** access one word at a time.
- A pointer (used by **lw** and **sw**) is just a memory address, we can add to it or subtract from it (using offset).
- Memory can be used for variables we can't store in registers, but 100x slower than using registers directly
 - Use loads and stores as infrequently as possible!
- New Instructions:
lw, sw, lb, sb, lbu